SpecBridge: Mid-Submission Progress Report

Unmanned Rooks | Hussain Lohawala | Qasim Naik | Tarun Rajai

1. Summary

Phase 1 is built and tested: golden tickets, a frozen workspace, a swappable model layer and a code library. Its hybrid search puts a really-changed file in the top 10 for **84% of golden tickets**. Phase 2 builds the agent that turns a Jira ticket into a Confluence page of code-grounded UAT test cases. Repository: github.com/H9660/SpecBridge.

2. Problem and scope

Scope is unchanged from the proposal. Before any code is written, SpecBridge reads a Jira ticket and the current frontend code, then writes UAT test cases (preconditions, action, expected result) to Confluence for the PM to review. The test bed is 5 connected OpenMRS 3 repos (React/TypeScript), each frozen at its 31 March 2026 commit. Every golden ticket was fixed after that date, so the real fix is the answer key. The code knowledge base (Approach 1) is built first. Live GitHub reading (Approach 2) moves into the Phase 2 comparison, because GitHub search only covers the latest code, which already holds the fixes.

3. Phase 1: milestones achieved

All four Phase 1 steps pass their smoke tests. Each step has a script that prints PASS, FAIL, WARN or PENDING per check and saves a report in `data/`.

Step	Built	Smoke test result
1. Golden tickets	268 post-cutoff tickets mined from OpenMRS Jira and git, 179 usable, 63 hand-picked; the files each fix changed are the answer key	All pass, 2 known warnings
2. Workspace and Jira	Repos cloned at frozen commits; 63 tickets copied to Jira `SPEC` with answer leaks removed; 6 adversarial tickets added (SPEC-64 to 69)	16 of 16 pass

Step	Built	Smoke test result
3. Model layer	`brain.py`: one interface for a cloud model (OpenRouter, default Gemini 3.8 Flash) or a local one (Ollama); schema-checked JSON, retries, cost per call	15 pass, 1 skipped
4. Code library	3,827 code pieces with on-screen text and app location; 6,128 links; keyword, meaning and link search in one SQLite file; 4 read-only agent tools	23 pass, 1 warning

Code search scores on 61 golden tickets (hit@10 = share of tickets with a changed file in the top 10):

Search method	All tickets	No file names in ticket	Vague tickets
Keyword, whole files	0.79	0.70	0.65
Keyword + links	0.80	0.74	0.59
Hybrid: keyword + meaning + links	**0.84**	**0.79**	**0.71**

4. Repository and development history

The code is plain Python 3.12 (standard library plus `numpy`) at github.com/H9660/SpecBridge. `specbridge/` holds the product code (model layer, code parser, code library and search, agent tools, scoring). `scripts/` holds setup, Jira import, scoring and one smoke test per step. `config/` and `data/` hold settings, frozen commits, golden tickets and test reports. `docs/` holds the build steps, architecture, decisions log and a map of every file.

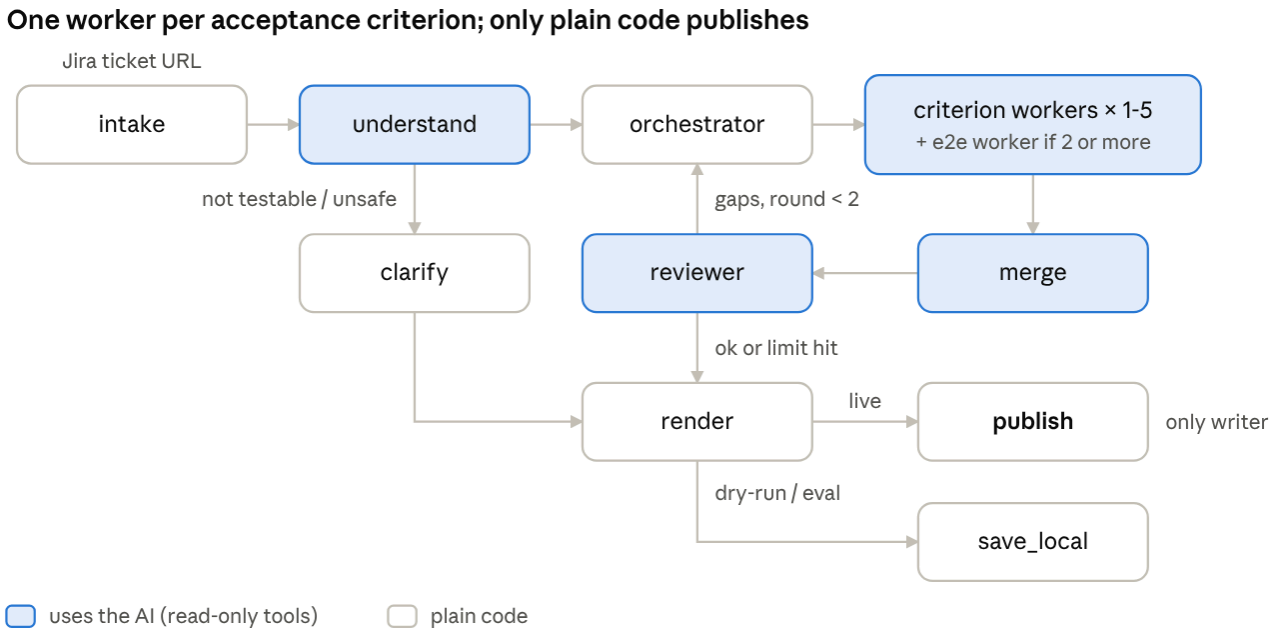
Commits, newest first (8 commits, including 1 merged pull request):

Date	Commit	Change
2026-11-04	`bdcddcf`	Agent architecture (LangGraph design, D1-D19, diagram)
2026-10-27	`b76d8de`, `649641e`	Merged pull request #1 and main
2026-10-27	`0549086`	Default model switched to Gemini 3.8 Flash
2026-10-27	`ae736a2`	Step 4: code library, search, links, agent tools

Date	Commit	Change
2026-10-26	`7822db2` , `b827055`	Build steps document; repo links and frozen versions
2026-10-26	`df01609`	Steps 1-3: golden tickets, workspace, Jira import, model layer

5. Agent architecture for Phase 2

The Phase 2 agent is designed in `docs/ARCHITECTURE.md` : a LangGraph graph with fixed outer steps, where the AI acts freely only inside each worker's search loop.



SpecBridge agent · LangGraph main graph

Each worker searches the code with the four Step 4 tools and cites the files and lines it read. The reviewer sends weak criteria back at most twice.

6. Phase 2: plan

Phase 2 follows the agent's run order, and each step again ends with a smoke test.

Step	To build	Done when
6. Ticket reader	`intake` (team Jira only, clean, flag injections), `understand` (criteria, screens, questions, risks), `clarify`	69 tickets read; 6 adversarial ones flagged

Step	To build	Done when
7. Agent	Graph and checkpointer, orchestrator, criterion and end-to-end workers, merge, reviewer, cost and round limits	10 golden tickets run end to end; every case cites code
8. Output	Page render, Confluence publish and Jira comment, CLI and local page, bot account	Re-runs keep PM edits; secrets block publishing
9. Scoring	Golden answers for 30-50 tickets, rubric, LLM judge, ship gate	Coverage, grounding, safety, cost scored
10. Improve	3+ rounds; compare 1 vs many workers, models, code library vs live GitHub	Each change logged against the last run
11. Users	5+ real users, demo, presentation	Feedback recorded

Proposed ship gate: coverage ≥ 90%, grounding ≥ 95%, 6 of 6 adversarial tickets safe, rubric ≥ 4/5, at most $0.10 and 3 minutes per ticket.

7. Risks and open questions

Risk	Mitigation
Search just under the 0.80 bar; cross-repo recall@20 only 0.50	Workers search again on their own; retrieval re-scored on the agent's evidence
Agent loops overrun cost and time	$0.25 per ticket, 2 review rounds, ~12 tool calls per worker; a hit ends "incomplete"
Prompt injection in tickets or code	Read-only AI tools, ticket text marked as data, only plain code publishes